

POLITÉCNICO GRANCOLOMBIANO

Facultad de Ingeniería y Ciencias Básicas

CONCEPTOS FUNDAMENTALES DE PROGRAMACIÓN

Trabajo Grupal — Subgrupo 7

ENTREGA 3 — PROYECTO FINAL Y SUSTENTACIÓN

Semanas 7 y 8

Integrantes:

Santiago Fuentes Lopez
Cristian Mauricio Ricardo Rojas
Lina Garnica Gómez

Repositorio GitHub:

<https://github.com/cr1c4rd0/CFDP>

Bogotá D.C., 2025

1. Descripción del Proyecto

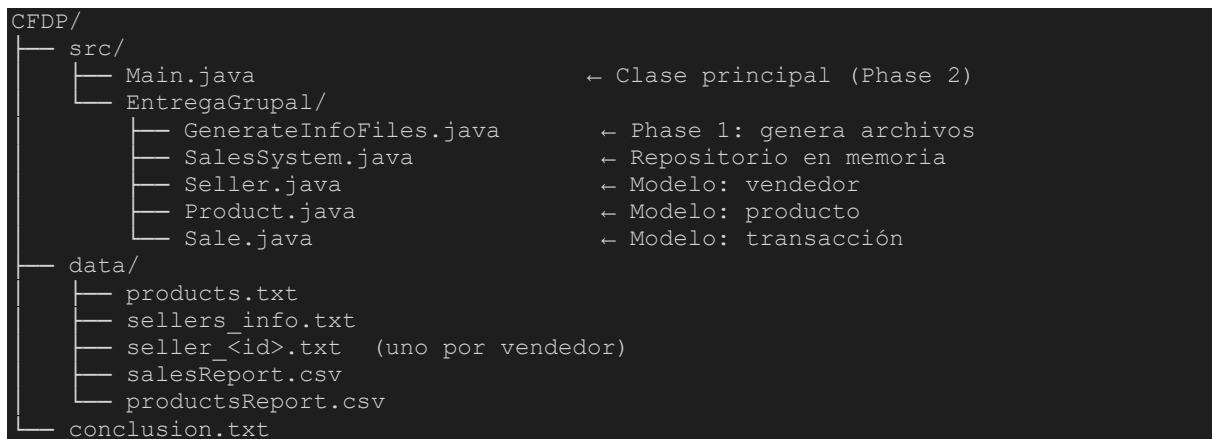
El proyecto CFDP (Conceptos Fundamentales de Programación) es un sistema de gestión de ventas desarrollado en Java como trabajo grupal de la asignatura. El objetivo es construir un programa que:

- Genere archivos planos con datos de vendedores, productos y ventas (GenerateInfoFiles).
- Lea esos archivos, los cargue en estructuras en memoria y produzca reportes CSV ordenados (Main).
- salesReport.csv: vendedores ordenados por total recaudado (descendente).
- productsReport.csv: productos ordenados por cantidad vendida (descendente).

La ejecución es completamente automática, sin intervención del usuario.

2. Estructura del Proyecto

El proyecto sigue la siguiente organización de directorios:



3. Documentación de Clases (Javadoc)

3.1 Seller.java

Representa a un vendedor del sistema. Almacena tipo de documento, número, nombre y apellido. Es inmutable: todos los campos se asignan en el constructor.

```
/**  
 * Represents a seller registered in the sales system.  
 * Each seller is identified by a document type and a unique document number.  
 *  
 * @author Subgrupo 7  
 * @version 3.0  
 */
```

```
public class Seller {
    private String documentType;    // CC, CE, TI, PA
    private long   documentNumber;  // Unique numeric ID
    private String firstName;
    private String lastName;

    /** Returns the full name: firstName + " " + lastName */
    public String getFullName() { return firstName + " " + lastName; }
}
```

3.2 Product.java

Representa un producto disponible. El precio se almacena como double para permitir valores fraccionarios en pesos colombianos.

```
/**
 * Represents a product available for sale.
 * Price is stored as double (Colombian pesos).
 *
 * @author Subgrupo 7
 * @version 3.0
 */
public class Product {
    private int    id;
    private String name;
    private double pricePerUnit; // COP
}
```

3.3 Sale.java

Representa una transacción de venta: asocia un vendedor con un producto y la cantidad vendida. El total se calcula a demanda.

```
/**
 * Represents a single sales transaction.
 * Total revenue = quantity * product.pricePerUnit
 *
 * @author Subgrupo 7
 * @version 3.0
 */
public class Sale {
    private Seller seller;
    private Product product;
    private int    quantity;

    /** Computes total revenue: quantity * pricePerUnit */
    public double getTotal() { return quantity * product.getPricePerUnit(); }
}
```

3.4 SalesSystem.java

Repositorio central en memoria. Contiene tres listas (vendedores, productos, ventas) y expone addX()/getX() para cada una. El método listData() imprime un resumen formateado en consola.

```
/**
 * Central repository holding all sellers, products and sales.
 * Acts as an in-memory data store.
 *
 * @author Subgrupo 7
```

```

* @version 3.0
*/
public class SalesSystem {
    private List<Seller>    sellers;
    private List<Product> products;
    private List<Sale>     sales;

    /** Prints formatted summary of all data to stdout. */
    public void listData() { ... }
}

```

3.5 GenerateInfoFiles.java

Genera los archivos de entrada con datos pseudoaleatorios: products.txt, sellers_info.txt y un seller_<id>.txt por cada vendedor. Este es el main de Phase 1.

```

/**
 * Generates flat files used as input for the sales system.
 * Phase 1 entry point.
 *
 * @author Subgrupo 7
 * @version 3.0
 */
public class GenerateInfoFiles {
    public static void main(String[] args) {
        createProductsFile(NUM_PRODUCTS);
        createSellersInfoFile(NUM_SELLERS, names, ids);
        for (int i = 0; i < NUM_SELLERS; i++)
            createSellerSalesFile(salesCount, names[i], ids[i]);
    }
}

```

3.6 Main.java — Clase Principal (Phase 2)

Orquesta las dos fases: llama a GenerateInfoFiles.main(), luego lee todos los archivos generados, los carga en SalesSystem y produce los dos reportes CSV ordenados. No requiere entrada del usuario.

```

/**
 * Entry point for the CFDP Sales System project.
 * Phase 1: calls GenerateInfoFiles.main(args)
 * Phase 2: reads files → loads SalesSystem → generates sorted CSV reports
 * - salesReport.csv: sellers by total revenue (desc)
 * - productsReport.csv: products by total quantity sold (desc)
 * No user input required.
 *
 * @author Subgrupo 7
 * @version 3.0
 */
public class Main {
    public static void main(String[] args) {
        GenerateInfoFiles.main(args); // Phase 1
        SalesSystem system = new SalesSystem();
        loadProducts(system); // Read products.txt
        loadSellers(system); // Read sellers_info.txt
        loadSales(system, sellers, productMap); // Read seller_*.txt
        system.listData(); // Print summary
        generateSalesReport(system); // salesReport.csv
        generateProductsReport(system, productMap); // productsReport.csv
    }
}

```

```
}
```

4. Estado Final del Proyecto

La siguiente tabla muestra el estado de cada componente requerido en la Entrega 3:

Componente	Estado	Notas
<code>Seller.java</code>	✓ Completo	Modelo inmutable con Javadoc completo
<code>Product.java</code>	✓ Completo	Modelo con precio double y Javadoc
<code>Sale.java</code>	✓ Completo	Calcula total en <code>getTotal()</code> , Javadoc
<code>SalesSystem.java</code>	✓ Completo	Repositorio con <code>listData()</code> documentado
<code>GenerateInfoFiles.java</code>	✓ Completo	Genera 3 tipos de archivos planos
<code>Main.java</code>	✓ Completo	Lee archivos y genera <code>salesReport.csv</code> y <code>productsReport.csv</code> ordenados
<code>salesReport.csv</code>	✓ Generado	Vendedores ordenados por revenue desc
<code>productsReport.csv</code>	✓ Generado	Productos ordenados por cantidad desc
<code>conclusion.txt</code>	✓ Incluido	Aprendizajes, aplicaciones y dificultades
Repositorio GitHub	✓ Publicado	https://github.com/cr1c4rd0/CFDP

5. Evidencia de Ejecución

5.1 Salida de consola — `Main.java` completo

La siguiente captura muestra la ejecución completa: Phase 1 (generación de archivos), carga de datos, `listData()` y generación de reportes.

Main.java – console output

```
=== CFDP Sales System – Phase 1: Generate files ===
Products file created: data/products.txt
Sellers info file created: data/sellers_info.txt
Sales file created for Carlos Hernandez -> data/seller_1848948063.txt
Sales file created for Isabella Ramirez -> data/seller_1588826095.txt
Sales file created for Sofia Gonzalez -> data/seller_1399366692.txt
Sales file created for Camila Garcia -> data/seller_1371408160.txt
Sales file created for Ana Vargas -> data/seller_1029073107.txt
All files generated successfully in: data/

=== CFDP Sales System – Phase 2: Read files and generate reports ===
Products loaded: 10
Sellers loaded: 5
Sales loaded: 31

=== SELLERS ===
PA;1848948063 - Carlos Hernandez
CC;1588826095 - Isabella Ramirez
PA;1399366692 - Sofia Gonzalez
PA;1371408160 - Camila Garcia
CC;1029073107 - Ana Vargas

=== PRODUCTS ===
1 - Cafe Tostado $22139
2 - Azucar Blanca $34316
3 - Arroz Diana $43310
4 - Leche Entera $6779
5 - Pan Tajado $29243
6 - Aceite Girasol $3028
7 - Sal Refinada $49212
8 - Harina de Trigo $48662
9 - Pasta Espagueti $42854
10 - Atun en Lata $10828

=== SALES ===
Carlos Hernandez sold 31 units of Azucar Blanca -> Total: $1,063,796
Carlos Hernandez sold 36 units of Arroz Diana -> Total: $1,559,160
Carlos Hernandez sold 97 units of Sal Refinada -> Total: $4,773,564
Carlos Hernandez sold 69 units of Leche Entera -> Total: $467,751
Carlos Hernandez sold 55 units of Atun en Lata -> Total: $595,540
Carlos Hernandez sold 15 units of Harina de Trigo -> Total: $729,930
Carlos Hernandez sold 71 units of Azucar Blanca -> Total: $2,436,436
Carlos Hernandez sold 31 units of Pasta Espagueti -> Total: $1,328,474
Isabella Ramirez sold 92 units of Azucar Blanca -> Total: $3,157,072
Isabella Ramirez sold 8 units of Azucar Blanca -> Total: $274,528
Isabella Ramirez sold 33 units of Pasta Espagueti -> Total: $1,414,182
Isabella Ramirez sold 40 units of Harina de Trigo -> Total: $1,946,480
Isabella Ramirez sold 16 units of Atun en Lata -> Total: $173,248
Sofia Gonzalez sold 77 units of Pasta Espagueti -> Total: $3,299,758
Sofia Gonzalez sold 58 units of Leche Entera -> Total: $393,182
Sofia Gonzalez sold 4 units of Pan Tajado -> Total: $116,972
Sofia Gonzalez sold 41 units of Arroz Diana -> Total: $1,775,710
Sofia Gonzalez sold 77 units of Sal Refinada -> Total: $3,789,324
Sofia Gonzalez sold 42 units of Pan Tajado -> Total: $1,228,206
Camila Garcia sold 46 units of Atun en Lata -> Total: $498,088
Camila Garcia sold 13 units of Atun en Lata -> Total: $140,764
Camila Garcia sold 30 units of Aceite Girasol -> Total: $90,840
Camila Garcia sold 21 units of Atun en Lata -> Total: $227,388
Camila Garcia sold 87 units of Azucar Blanca -> Total: $2,985,492
Camila Garcia sold 6 units of Leche Entera -> Total: $40,674
Camila Garcia sold 84 units of Sal Refinada -> Total: $4,133,808
Ana Vargas sold 54 units of Pan Tajado -> Total: $1,579,122
Ana Vargas sold 70 units of Pasta Espagueti -> Total: $2,999,780
Ana Vargas sold 17 units of Leche Entera -> Total: $115,243
Ana Vargas sold 3 units of Azucar Blanca -> Total: $102,948
Ana Vargas sold 24 units of Atun en Lata -> Total: $259,872
```

Fig. 1 — Ejecución completa de Main.java (Phase 1 + Phase 2 + listData)

5.2 salesReport.csv — Vendedores por total recaudado

Reporte generado en data/salesReport.csv. Formato: NombreVendedor;TotalRecaudado. Ordenado de mayor a menor recaudo.

```
salesReport.csv

salesReport.csv generated -> data/salesReport.csv

Carlos Hernandez      $12,954,651
Sofia Gonzalez        $10,603,152
Camila Garcia         $8,117,054
Isabella Ramirez      $6,965,510
Ana Vargas            $5,056,965

# Contents of data/salesReport.csv (sorted by revenue desc):

Carlos Hernandez;12954651
Sofia Gonzalez;10603152
Camila Garcia;8117054
Isabella Ramirez;6965510
Ana Vargas;5056965

Process finished with exit code 0
```

Fig. 2 — salesReport.csv generado por Main.java

5.3 productsReport.csv — Productos por cantidad vendida

Reporte generado en data/productsReport.csv. Formato: NombreProducto;PrecioPorUnidad. Ordenado de mayor a menor cantidad vendida.

```
productsReport.csv

productsReport.csv generated -> data/productsReport.csv

Azucar Blanca           $34,316   [292 units sold]
Sal Refinada            $49,212   [258 units sold]
Pasta Espagueti        $42,854   [211 units sold]
Atun en Lata           $10,828   [175 units sold]
Leche Entera           $6,779    [150 units sold]
Pan Tajado              $29,243   [100 units sold]
Arroz Diana            $43,310   [77 units sold]
Harina de Trigo         $48,662   [55 units sold]
Aceite Girasol          $3,028    [30 units sold]
Cafe Tostado           $22,139   [0 units sold]

# Contents of data/productsReport.csv (sorted by quantity desc):

Azucar Blanca;34316
Sal Refinada;49212
Pasta Espagueti;42854
Atun en Lata;10828
Leche Entera;6779
Pan Tajado;29243
Arroz Diana;43310
Harina de Trigo;48662
Aceite Girasol;3028
Cafe Tostado;22139

Process finished with exit code 0
```

Fig. 3 — productsReport.csv generado por Main.java

6. Conclusión

6.1 ¿Qué aprendimos?

A lo largo de este proyecto consolidamos los fundamentos de la programación orientada a objetos en Java: encapsulamiento, cohesión de clases y separación de responsabilidades. Aprendimos a diseñar un sistema desde cero partiendo de clases modelo (Seller, Product, Sale), una clase repositorio (SalesSystem) y dos clases ejecutables con método main (GenerateInfoFiles y Main), respetando una arquitectura limpia donde cada clase tiene un propósito único y bien definido.

En el lado práctico dominamos la lectura y escritura de archivos planos con `BufferedReader` y `BufferedWriter`, el uso de colecciones (`ArrayList`, `HashMap`) para agregar datos en memoria, y el ordenamiento con `Comparator` para producir reportes ordenados por criterios de negocio. También practicamos la documentación formal del código mediante Javadoc, un estándar industrial que facilita el mantenimiento futuro.

6.2 Aplicaciones Profesionales

- ETL (Extract-Transform-Load): la lectura de archivos planos y su carga en estructuras en memoria es exactamente lo que hacen los pipelines de datos empresariales antes de insertar registros en una base de datos relacional.
- Microservicios de reportes: la generación de los CSV es equivalente a un endpoint `/reports` que devuelve datos agregados para dashboards de ventas.
- Separación de capas: la división entre `GenerateInfoFiles` (entrada de datos) y `Main` (procesamiento y salida) anticipa el patrón MVC y la arquitectura en capas de aplicaciones enterprise.
- Mantenibilidad: documentar con Javadoc y mantener métodos pequeños con responsabilidad única reduce el costo de incorporar nuevos integrantes al equipo.

6.3 Dificultades Encontradas

- Lectura robusta de archivos: el formato de los archivos de ventas incluía un punto y coma al final de cada línea (ej: `"3;45;"`), lo que requirió manejo especial al hacer split para evitar arreglos con elementos vacíos.
- Coherencia de datos: al cargar las ventas fue necesario validar que el ID del producto referenciado existiera en el mapa, agregando mensajes de advertencia en lugar de lanzar excepciones que detuvieran el sistema.
- Formato de salida: unificar el formato numérico entre consola y CSV requirió uso de `String.format` y ajustes en los tipos de datos.
- Coordinación en equipo: dividir el trabajo en clases independientes facilitó el desarrollo en paralelo, pero exigió definir contratos claros (firmas de métodos, formatos de archivo) antes de comenzar.

7. Repositorio GitHub

El código fuente completo, incluyendo todos los archivos Java, los datos de prueba y el `conclusion.txt`, se encuentra disponible en:

<https://github.com/cr1c4rd0/CFDP>